

Pandas

1. Overview

Pandas is an open-source Python library for working with labeled, tabular, and time-series data. Its two central data structures are:

- **Series**: a one-dimensional labeled array.
- **DataFrame**: a two-dimensional table whose rows and columns have labels.

Pandas sits between raw data sources and modeling libraries. In a typical machine-learning workflow, it is used to load data, inspect quality, clean missing or invalid values, join multiple sources, engineer features, aggregate observations, create time-aware datasets, and export model-ready tables.

```
CSV / SQL / Parquet / API
    |
    v
  Pandas
validate -> clean -> join -> transform -> aggregate
    |
    v
NumPy / scikit-learn / PyTorch / feature store / dashboard
```

Real-world uses include:

- Building customer-level features from transaction logs.
- Combining labels, user profiles, and event histories.
- Preparing train, validation, and test datasets.
- Analyzing model predictions and errors by subgroup.
- Processing timestamps for forecasting and monitoring.
- Producing offline features later reproduced in an online feature pipeline.

Pandas is optimized for **in-memory analytical workloads** on a single machine. It is not a distributed database, stream processor, or model-training framework. When data becomes larger than memory or execution must be distributed, tools such as SQL engines, Spark, Polars, Dask, or DuckDB may be more appropriate.

2. Intuition

Think of a **DataFrame** as a spreadsheet with a programmable interface and strict alignment rules.

Suppose an e-commerce table contains one row per order:

order_id	customer_id	amount	order_time
101	7	499.0	2026-01-02

order_id	customer_id	amount	order_time
102	8	250.0	2026-01-03
103	7	100.0	2026-01-05

With Pandas, a few operations can answer:

- Which orders are above ₹300? — boolean filtering.
- How much did each customer spend? — grouping and aggregation.
- What was each customer's previous order? — grouped shifting.
- Which orders have no amount? — missing-value detection.
- Which customer attributes belong to each order? — merging.

The most important intuition is **label alignment**. Pandas often matches data by index or column label rather than merely by physical position. This is powerful, but accidental index mismatches are a common source of silent missing values.

```
import pandas as pd

left = pd.Series([10, 20], index=["a", "b"])
right = pd.Series([1, 2], index=["b", "c"])

print(left + right)
# a      NaN    # no matching label in right
# b     21.0    # 20 + 1
# c      NaN    # no matching label in left
```

Pandas operations are usually **vectorized**: one expression describes work over an entire column rather than writing a Python loop for each row.

3. Prerequisites

Before studying Pandas, know the following:

Programming prerequisites

- Python variables, lists, dictionaries, functions, and exceptions.
- Slicing and boolean expressions.
- Basic NumPy arrays and data types are helpful.
- Familiarity with files such as CSV and JSON.

Data and statistics prerequisites

- Rows as observations and columns as variables/features.
- Numerical, categorical, ordinal, datetime, and text data.
- Mean, median, variance, standard deviation, quantiles, and correlation.
- Missing data and outliers.

- Basic relational concepts: keys, one-to-one, one-to-many, and many-to-many joins.

ML prerequisites

- Features (x) and targets (y).
- Train/validation/test splits.
- Data leakage.
- Preprocessing and feature engineering.

No deep-learning or advanced mathematics is required.

4. Core Concepts

4.1 `Series` and `DataFrame`

What they mean

A `Series` stores values plus an index. A `DataFrame` is a collection of aligned `Series` objects sharing a row index.

```
import pandas as pd

scores = pd.Series([82, 91, 76], name="score", index=["A", "B", "C"])

students = pd.DataFrame({
    "name": ["Asha", "Ravi", "Meera"],
    "score": [82, 91, 76],
    "placed": [True, True, False],
})
```

Why they matter

Labels make operations expressive: `students["score"]` identifies a feature by name, while the index identifies observations.

Simple example

```
students["score"].mean()    # 83.0
students.shape              # (3, 3)
students.columns.tolist()   # ['name', 'score', 'placed']
```

Common interview angle

Question: What is the difference between a `Series` and a one-column `DataFrame`?

Answer: A **Series** is one-dimensional and has a **name**; a one-column **DataFrame** is two-dimensional and has a column index. Selecting `df["score"]` returns a **Series**, whereas `df[["score"]]` returns a **DataFrame**.

4.2 Index and label alignment

What it means

Every row has an index label. Arithmetic, assignment, joins, and concatenation may align objects using these labels.

Why it matters

Alignment prevents records from being combined merely because they happen to occupy the same position. However, stale or duplicate indexes can create unexpected **NaN** values or duplicated output.

```
df = pd.DataFrame({"x": [10, 20]}, index=[100, 200])
s = pd.Series([1, 2], index=[200, 100])
df["x_plus_s"] = df["x"] + s
# index 100: 10 + 2; index 200: 20 + 1
```

Common interview angle

Explain why `df["new"] = pd.Series([...])` can produce missing values: the new **Series** index may not match `df.index`. Use a matching index, assign a NumPy array for positional semantics, or reset indexes deliberately.

4.3 Data types (**dtype**)

What it means

Each column has a data type. Common types include integers, floating-point values, booleans, strings, categories, and datetimes.

```
df = pd.DataFrame({
    "age": pd.Series([21, None, 23], dtype="Int64"),
    "city": pd.Series(["Pune", "Delhi", "Pune"], dtype="category"),
    "joined": pd.to_datetime(["2025-01-01", "2025-02-03", None]),
})
print(df.dtypes)
```

Why it matters

Data types affect memory, speed, missing-value behavior, comparisons, serialization, and compatibility with ML libraries.

- Plain NumPy **int64** cannot represent **NaN**.

- Nullable Pandas `Int64` can represent missing integers with `pd.NA`.
- `category` can reduce memory for repeated low-cardinality strings.
- Datetime types enable `.dt` operations and time-based resampling.

Common interview angle

Why is an apparently numeric column stored as `object`/string? Typical causes are currency symbols, commas, mixed values, or invalid tokens. Clean and convert with `pd.to_numeric(..., errors="coerce")`, then inspect newly introduced missing values.

4.4 Inspection and descriptive statistics

Important inspection methods:

```
df.head()
df.tail()
df.sample(5, random_state=42)
df.shape
df.info()
df.describe(include="all")
df.nunique(dropna=False)
df.isna().sum()
df.duplicated().sum()
```

Why it matters

Inspection catches schema problems before they contaminate downstream features. `info()` is especially useful for row count, non-null count, dtypes, and approximate memory use.

Common interview angle

Describe the first checks after loading a dataset: shape, sample records, schema/dtypes, missingness, duplicates, target balance, impossible ranges, and unique-key constraints.

4.5 Selecting rows and columns

Pandas provides several selectors:

Selector	Meaning	End-point behavior
<code>df["col"]</code>	One column as <code>Series</code>	Not applicable
<code>df[["a", "b"]]</code>	Columns as <code>DataFrame</code>	Not applicable
<code>df.loc[rows, cols]</code>	Label-based selection	Label slice is usually inclusive
<code>df.iloc[rows, cols]</code>	Integer-position selection	Python-style end exclusive
<code>df.at[row, col]</code>	One scalar by label	Scalar only
<code>df.iat[row, col]</code>	One scalar by position	Scalar only

```
df.loc[df["score"] >= 80, ["name", "score"]]  
df.iloc[:2, 0:2]
```

Common interview angle

The difference between `.loc` and `.iloc` is a favorite question. `.loc` uses labels; `.iloc` uses zero-based integer positions.

4.6 Boolean filtering and query logic

```
selected = df.loc[  
    (df["score"] >= 80) & df["city"].isin(["Pune", "Delhi"])  
]  
  
missing_or_invalid = df.loc[df["salary"].isna() | (df["salary"] < 0)]
```

Use `&`, `|`, and `~` for element-wise AND, OR, and NOT. Parenthesize each comparison because Python operator precedence can otherwise change the expression.

`query()` can improve readability for simple conditions:

```
threshold = 80  
df.query("score >= @threshold and city == 'Pune'")
```

Common interview angle

Python's `and` and `or` expect one truth value and therefore cannot combine entire boolean `Series` objects. Use `&` and `|`.

4.7 Missing data

Pandas may represent missing values using `NaN`, `NaT`, or `pd.NA`, depending on dtype.

```
df.isna()                                # boolean mask  
df.isna().mean()                         # missing fraction by column  
df.dropna(subset=["target"])             # remove rows without labels  
df["age"].fillna(df["age"].median())  
df["city"].fillna("Unknown")  
df["value"].interpolate(method="linear")
```

Why it matters

Missingness may carry information, but careless imputation can bias data or leak validation information.

Simple example

For a training set, compute the median only on training data and reuse it for validation/test data:

```
median_age = train["age"].median()
train["age"] = train["age"].fillna(median_age)
valid["age"] = valid["age"].fillna(median_age)
```

Common interview angle

Do not say “always fill missing values with the mean.” The correct method depends on the missingness mechanism, distribution, model, and business meaning. Preserve a missingness indicator when absence itself may be predictive.

4.8 Sorting, duplicates, and uniqueness

```
df.sort_values(["customer_id", "timestamp"], ascending=[True, False])
df.sort_index()
df.duplicated(subset=["order_id"], keep=False)
df.drop_duplicates(subset=["order_id"], keep="last")
df["order_id"].is_unique
```

Common interview angle

Before dropping duplicates, define what makes a record duplicate and which record is authoritative. `keep="last"` is meaningful only if rows have first been sorted by a trustworthy update timestamp.

4.9 Vectorized transformations

```
df["log_income"] = np.log1p(df["income"].clip(lower=0))
df["is_senior"] = df["experience_years"].ge(5)
df["email_domain"] = df["email"].str.extract(r"@[^@]+$", expand=False)
```

Prefer vectorized column operations over row iteration. Use `map` for element-wise mappings, `apply` when a suitable vectorized operation does not exist, and avoid `iterrows()` in performance-sensitive transformation code.

```
label_map = {"low": 0, "medium": 1, "high": 2}
df["priority_code"] = df["priority"].map(label_map)
```

Common interview angle

`apply()` is not automatically vectorized; it often calls a Python function once per row or column. Built-in arithmetic, string, datetime, groupby, and NumPy operations are generally faster and clearer.

4.10 Grouping and aggregation

The split-apply-combine pattern:

1. **Split** rows into groups.
2. **Apply** an aggregation or transformation.
3. **Combine** results.

```
summary = (
    orders.groupby("customer_id", as_index=False)
    .agg(
        total_spend=("amount", "sum"),
        mean_spend=("amount", "mean"),
        order_count=("order_id", "nunique"),
        last_order=("order_time", "max"),
    )
)
```

Important group operations:

- `agg`: reduces each group to summary rows.
- `transform`: returns output aligned to the original rows.
- `filter`: keeps or removes entire groups.
- `apply`: flexible but often slower and harder to reason about.

```
orders["customer_mean"] = orders.groupby("customer_id")
["amount"].transform("mean")
orders["amount_vs_mean"] = orders["amount"] - orders["customer_mean"]
```

Common interview angle

`agg` changes the number of rows; `transform` normally preserves the original row count and index, making it suitable for adding group-level features to each record.

4.11 Merging and joining

```
enriched = orders.merge(
    customers,
    on="customer_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)
```

Join	Rows retained
<code>inner</code>	Matching keys from both sides
<code>left</code>	All left rows plus right matches
<code>right</code>	All right rows plus left matches
<code>outer</code>	Union of keys from both sides
<code>cross</code>	Cartesian product

Why it matters

Most production datasets are assembled from multiple tables. An incorrect many-to-many join can multiply rows and corrupt aggregates or training weights.

Common interview angle

Use `validate="one_to_one"`, `"one_to_many"`, `"many_to_one"`, or `"many_to_many"` to assert expected key cardinality. Use `indicator=True` to audit unmatched rows.

4.12 Concatenation, reshaping, and pivoting

```
all_months = pd.concat([jan, feb, mar], ignore_index=True)

wide = sales.pivot(index="date", columns="region", values="revenue")

pivot = sales.pivot_table(
    index="region",
    columns="product",
    values="revenue",
    aggfunc="sum",
    fill_value=0,
)

long = wide.reset_index().melt(
    id_vars="date",
    var_name="region",
    value_name="revenue",
)
```

`pivot()` requires each index-column pair to be unique. `pivot_table()` can aggregate duplicates.

Common interview angle

Explain wide versus long data. Long form is often easier for grouping, plotting, and tidy transformations; wide form can be convenient for matrices and reports.

4.13 Datetime and time-series operations

```

events["timestamp"] = pd.to_datetime(events["timestamp"], utc=True,
errors="coerce")
events = events.sort_values("timestamp")
events["day_of_week"] = events["timestamp"].dt.dayofweek
events["hour"] = events["timestamp"].dt.hour

daily = (
    events.set_index("timestamp")
    .resample("1D")["value"]
    .sum()
)

```

Useful operations include:

- Parsing with `pd.to_datetime`.
- Components through `.dt`.
- Time-based grouping with `resample`.
- Window statistics with `rolling`, `expanding`, and exponentially weighted methods.
- Lags with `shift`.
- Time-aware joins with `merge_asof`.

```

events["previous_value"] = events.groupby("user_id")["value"].shift(1)
events["rolling_mean_7"] = (
    events.groupby("user_id")["value"]
    .transform(lambda s: s.shift(1).rolling(7, min_periods=1).mean())
)

```

The `shift(1)` prevents the current observation from entering a historical feature.

Common interview angle

Time-series splits must preserve chronology. Random splitting can train on the future and evaluate on the past.

4.14 Categorical data and encoding

```

df["city"] = df["city"].astype("category")
encoded = pd.get_dummies(df, columns=["city"], dtype=int)

```

For model pipelines, prefer fitting an encoder on the training set through scikit-learn when validation/test categories must be handled consistently. `pd.get_dummies` independently applied to each split can produce different columns.

Common interview angle

Never integer-encode an unordered category and assume a distance-based model will ignore the artificial order. Use one-hot encoding or a model/encoder designed for categorical features.

4.15 Copying, views, and safe assignment

Chained selection can make assignment ambiguous:

```
# Avoid:
df[df["score"] < 0]["score"] = 0

# Prefer one explicit assignment:
df.loc[df["score"] < 0, "score"] = 0
```

Use `.copy()` when creating an intentionally independent working subset:

```
subset = df.loc[df["active"], ["user_id", "score"]].copy()
subset["score"] = subset["score"].clip(lower=0)
```

Common interview angle

The issue is not merely suppressing a warning. The problem is that chained indexing may operate on a temporary object rather than performing an unambiguous update to the original table.

4.16 Input/output and storage formats

```
df = pd.read_csv(
    "orders.csv",
    usecols=["order_id", "customer_id", "amount", "order_time"],
    dtype={"order_id": "int64", "customer_id": "int64"},
    parse_dates=["order_time"],
)

df.to_parquet("orders_clean.parquet", index=False)
```

Common formats:

Format	Strength	Limitation
CSV	Universal and human-readable	No reliable schema; larger and slower
Parquet	Columnar, compressed, typed	Requires a compatible engine
JSON	Natural for nested/web data	Verbose; orientations vary
SQL	Query data at source	Requires database and query design
Pickle	Preserves Python objects	Python-specific; unsafe for untrusted files

Common interview angle

Parquet is generally better for analytical pipelines because it preserves types, compresses well, and supports column-oriented access. CSV remains useful for interchange and small human-readable files.

5. Algorithm / Working Process

Pandas is a data-processing library rather than one algorithm. A reliable Pandas workflow follows a sequence of explicit contracts.

Step 1: Define the observation unit

Decide what one row represents: one customer, one transaction, one image, one patient visit, or one timestamp. Mixing observation levels creates duplicate labels and leakage.

Step 2: Load only required data

Read the needed columns, specify known dtypes, parse dates, and use chunking or source-side filtering when data is large.

Step 3: Inspect and validate

Check:

- Row and column counts.
- Column names and dtypes.
- Missingness and duplicates.
- Key uniqueness.
- Valid numerical ranges.
- Category values.
- Timestamp boundaries.
- Target distribution.

Step 4: Clean without destroying information

- Normalize column names and string values.
- Convert types explicitly.
- Handle missing values according to meaning.
- Deduplicate using business keys and ordering rules.
- Flag or cap invalid values only with domain justification.

Step 5: Join tables with cardinality checks

Choose join keys and join type deliberately. Assert the relationship using `validate` and audit unmatched rows.

Step 6: Split before learned preprocessing

Create training, validation, and test partitions before estimating imputation values, vocabularies, normalization statistics, or target-derived features.

Step 7: Engineer leakage-safe features

Use vectorized operations, grouped aggregations, windows, and lags. For prediction at time t , use only information available at or before t —and often strictly before t .

Step 8: Produce model inputs and metadata

Separate:

- Identifier columns.
- Feature matrix X .
- Target vector y .
- Timestamp or split metadata.

Step 9: Verify invariants

Examples:

```
assert features["customer_id"].is_unique
assert features["target"].notna().all()
assert set(valid.columns) == set(train.columns)
assert np.isfinite(model_matrix.to_numpy()).all()
```

Step 10: Save in a typed format

Export model-ready data and preserve transformation logic in version-controlled code. Prefer Parquet for typed analytical data and use a model pipeline for fitted transformations.

6. Mathematical Foundation

Pandas implements data manipulations, so it has no universal loss function or optimizer. Its mathematical foundation comes from arrays, relational algebra, descriptive statistics, and window operations.

6.1 Labeled data model

A **Series** can be viewed as a mapping from an index set to values:

$$s : I \rightarrow V$$

For two series s and t , aligned addition operates on the union of labels:

$$(s + t)_i = s_i + t_i, \quad i \in I_s \cap I_t$$

Labels absent from either side normally produce a missing result. A fill value can define alternative behavior.

6.2 Aggregation

For values $x_1, \dots, x_n$:

Mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Sample variance

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Pandas uses `ddof=1` by default for `Series.var()` and `Series.std()`, corresponding to sample variance. NumPy commonly defaults to `ddof=0`, so results may differ.

Weighted mean, implemented manually:

$$\bar{x}_w = \frac{\sum_i w_i x_i}{\sum_i w_i}$$

```
weighted_mean = (df["value"] * df["weight"]).sum() / df["weight"].sum()
```

6.3 Grouped aggregation

Let $g(i)$ denote the group of row i . A group mean feature is:

$$\mu_k = \frac{1}{|G_k|} \sum_{i:g(i)=k} x_i$$

`groupby(...).agg("mean")` produces one value per group, whereas `transform("mean")` broadcasts $\mu_{g(i)}$ back to every original row.

6.4 Standardization and min-max scaling

Z-score standardization:

$$z_i = \frac{x_i - \mu_{train}}{\sigma_{train}}$$

Min-max scaling:

$$x'_i = \frac{x_i - x_{min,train}}{x_{max,train} - x_{min,train}}$$

The subscript `train` is critical: statistics must be estimated from training data only. In real ML pipelines, use a fitted scikit-learn transformer rather than manually maintaining these numbers.

6.5 Covariance and correlation

Sample covariance between X and Y :

$$\text{cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

Pearson correlation:

$$r_{XY} = \frac{\text{cov}(X, Y)}{s_X s_Y}, \quad -1 \leq r \leq 1$$

```
numeric_corr = df.select_dtypes("number").corr(method="pearson")
rank_corr = df.select_dtypes("number").corr(method="spearman")
```

Correlation does not prove causation, can miss nonlinear dependence, and can be distorted by outliers or data leakage.

6.6 Rolling windows

For a trailing window of size w , the rolling mean at time t is:

$$m_t = \frac{1}{w} \sum_{j=0}^{w-1} x_{t-j}$$

For predictive features, one often uses only previous observations:

$$m_t^{\text{past}} = \frac{1}{w} \sum_{j=1}^w x_{t-j}$$

which corresponds to `s.shift(1).rolling(w).mean()`.

6.7 Exponentially weighted mean

An exponentially weighted moving average can be written as:

$$m_t = \alpha x_t + (1 - \alpha) m_{t-1}, \quad 0 < \alpha \leq 1$$

Recent observations receive more weight. In Pandas:

```
df["ewm"] = df["value"].ewm(alpha=0.3, adjust=False).mean()
```

6.8 Missingness rate

For missing indicator $M_i = 1$ if x_i is missing and 0 otherwise:

$$\text{missing rate} = \frac{1}{n} \sum_{i=1}^n M_i$$

This is exactly the intuition behind `df.isna().mean()`.

6.9 Relational algebra

A join combines relations based on key equality. For tables A and B joined on key k :

$$A \bowtie B = \{(a, b) : a.k = b.k\}$$

If key value v occurs m_v times in **A** and n_v times in **B**, an inner join creates:

$$m_v n_v$$

rows for that key. This explains why unintended many-to-many joins can explode row counts.

6.10 Loss function and optimization

These are **not applicable to Pandas itself**. Pandas does not learn parameters by minimizing a loss. It prepares and analyzes data consumed by models that do. Interview candidates should state this clearly rather than inventing a “Pandas training process.”

7. Practical Implementation

The following end-to-end example creates transaction data, validates it, builds leakage-safe customer features, trains a small classifier, and analyzes predictions. The model is included to demonstrate Pandas in a realistic ML workflow.

```
from __future__ import annotations

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

def make_data(seed: int = 42) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Create reproducible customer and transaction tables."""
    rng = np.random.default_rng(seed)
    n_customers = 500

    customers = pd.DataFrame({
        "customer_id": np.arange(1, n_customers + 1),
        "age": rng.integers(18, 66, n_customers).astype(float),
        "city": rng.choice(["Delhi", "Pune", "Bengaluru", "Chennai"],
n_customers),
        "signup_date": pd.Timestamp("2024-01-01")
        + pd.to_timedelta(rng.integers(0, 600, n_customers), unit="D"),
    })

    # Introduce realistic missing values.
    customers.loc[rng.choice(n_customers, 25, replace=False), "age"] = np.nan

    n_transactions = 5_000
    transactions = pd.DataFrame({
        "transaction_id": np.arange(1, n_transactions + 1),
        "customer_id": rng.integers(1, n_customers + 1, n_transactions),
```

```

        "timestamp": pd.Timestamp("2025-01-01")
        + pd.to_timedelta(rng.integers(0, 365, n_transactions), unit="D"),
        "amount": rng.gamma(shape=2.0, scale=700.0, size=n_transactions),
        "channel": rng.choice(["web", "app", "store"], n_transactions),
    })

# Churn is synthetic, but depends partly on customer attributes.
churn_probability = np.where(customers["city"].eq("Delhi"), 0.35, 0.18)
customers["churned"] = rng.binomial(1, churn_probability)
return customers, transactions

def validate_inputs(customers: pd.DataFrame, transactions: pd.DataFrame) -> None:
    """Fail early when table contracts are broken."""
    required_customer_cols = {"customer_id", "age", "city", "signup_date",
                              "churned"}
    required_transaction_cols = {
        "transaction_id", "customer_id", "timestamp", "amount", "channel"
    }

    missing_customer_cols = required_customer_cols - set(customers.columns)
    missing_transaction_cols = required_transaction_cols -
set(transactions.columns)
    if missing_customer_cols or missing_transaction_cols:
        raise ValueError(
            f"Missing columns: customers={missing_customer_cols}, "
            f"transactions={missing_transaction_cols}"
        )
    if not customers["customer_id"].is_unique:
        raise ValueError("customer_id must be unique in customers")
    if not transactions["transaction_id"].is_unique:
        raise ValueError("transaction_id must be unique in transactions")
    if transactions["amount"].lt(0).any():
        raise ValueError("amount cannot be negative")

def build_customer_features(
    customers: pd.DataFrame,
    transactions: pd.DataFrame,
    cutoff: pd.Timestamp,
) -> pd.DataFrame:
    """Build features using only transactions strictly before cutoff."""
    history = transactions.loc[transactions["timestamp"] < cutoff].copy()

    transaction_features = (
        history.groupby("customer_id", as_index=False)
        .agg(
            transaction_count=("transaction_id", "nunique"),
            total_spend=("amount", "sum"),
            average_spend=("amount", "mean"),
            spend_std=("amount", "std"),
            last_transaction=("timestamp", "max"),
        )
    )

```

```

app_share = (
    history["channel"].eq("app")
    .groupby(history["customer_id"])
    .mean()
    .rename("app_share")
    .reset_index()
)

features = (
    customers.merge(
        transaction_features,
        on="customer_id",
        how="left",
        validate="one_to_one",
    )
    .merge(app_share, on="customer_id", how="left", validate="one_to_one")
)

features["customer_tenure_days"] = (cutoff - features["signup_date"]).dt.days
features["recency_days"] = (cutoff - features["last_transaction"]).dt.days

zero_for_no_history = ["transaction_count", "total_spend", "app_share"]
features[zero_for_no_history] = features[zero_for_no_history].fillna(0)
return features

def split_by_customer(
    features: pd.DataFrame,
    seed: int = 42,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """Create reproducible 70/15/15 customer-level partitions."""
    shuffled = features.sample(frac=1, random_state=seed).reset_index(drop=True)
    train_end = int(len(shuffled) * 0.70)
    valid_end = int(len(shuffled) * 0.85)
    return (
        shuffled.iloc[:train_end].copy(),
        shuffled.iloc[train_end:valid_end].copy(),
        shuffled.iloc[valid_end:].copy(),
    )

def train_and_evaluate(
    train: pd.DataFrame,
    valid: pd.DataFrame,
) -> tuple[Pipeline, pd.DataFrame]:
    numeric_features = [
        "age",
        "transaction_count",
        "total_spend",
        "average_spend",
        "spend_std",
        "app_share",
        "customer_tenure_days",
    ]

```

```

        "recency_days",
    ]
    categorical_features = ["city"]
    target = "churned"

    numeric_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
        ("scaler", StandardScaler()),
    ])
    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("one_hot", OneHotEncoder(handle_unknown="ignore")),
    ])

    preprocessing = ColumnTransformer([
        ("numeric", numeric_pipeline, numeric_features),
        ("categorical", categorical_pipeline, categorical_features),
    ])

    model = Pipeline([
        ("preprocessing", preprocessing),
        ("classifier", LogisticRegression(max_iter=1_000,
class_weight="balanced")),
    ])

    feature_columns = numeric_features + categorical_features
    model.fit(train[feature_columns], train[target])

    probability = model.predict_proba(valid[feature_columns])[:, 1]
    prediction = (probability >= 0.5).astype(int)

    scored = valid[["customer_id", target, "city"]].assign(
        churn_probability=probability,
        prediction=prediction,
    )

    print(f"Validation ROC-AUC: {roc_auc_score(scored[target], probability):.3f}")
    print(classification_report(scored[target], prediction, zero_division=0))

    # Pandas makes subgroup error analysis concise.
    subgroup_report = (
        scored.assign(correct=scored[target].eq(scored["prediction"]))
        .groupby("city", as_index=False)
        .agg(
            customers=("customer_id", "size"),
            churn_rate=(target, "mean"),
            mean_prediction=("churn_probability", "mean"),
            accuracy=("correct", "mean"),
        )
        .sort_values("customers", ascending=False)
    )
    print(subgroup_report)
    return model, scored

```

```
def main() -> None:
    customers, transactions = make_data()
    validate_inputs(customers, transactions)

    features = build_customer_features(
        customers,
        transactions,
        cutoff=pd.Timestamp("2026-01-01"),
    )
    train, valid, test = split_by_customer(features)

    # Basic partition checks.
    assert set(train["customer_id"]).isdisjoint(valid["customer_id"])
    assert set(train["customer_id"]).isdisjoint(test["customer_id"])
    assert len(train) + len(valid) + len(test) == len(features)

    model, validation_predictions = train_and_evaluate(train, valid)
    print(validation_predictions.head())

if __name__ == "__main__":
    main()
```

Additional practical patterns

Reading a large CSV in chunks

```
totals = []
for chunk in pd.read_csv(
    "transactions.csv",
    usecols=["customer_id", "amount"],
    chunksize=250_000,
):
    totals.append(chunk.groupby("customer_id")["amount"].sum())

customer_totals = pd.concat(totals).groupby(level=0).sum()
```

Auditing a join

```
joined = orders.merge(
    customers,
    on="customer_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)
```

```
unmatched_rate = joined["_merge"].ne("both").mean()
if unmatched_rate > 0.01:
    raise ValueError(f"Too many unmatched orders: {unmatched_rate:.1%}")
```

Memory optimization

```
df["customer_id"] = pd.to_numeric(df["customer_id"], downcast="unsigned")
df["amount"] = pd.to_numeric(df["amount"], downcast="float")
df["channel"] = df["channel"].astype("category")

memory_mb = df.memory_usage(deep=True).sum() / 1024**2
print(f"Memory: {memory_mb:.2f} MB")
```

Downcasting can reduce precision or range, so validate limits before using it.

8. Code Explanation

Data generation

`make_data()` creates two tables at different granularities:

- `customers`: one row per customer, containing attributes and the target.
- `transactions`: many rows per customer, containing event history.

This mirrors a common production schema. A seeded NumPy generator makes the example reproducible.

Input validation

`validate_inputs()` checks schema, primary-key uniqueness, and domain constraints. Failing early is better than discovering corrupted features after training. These assertions are examples of **data contracts**.

Feature cutoff

`build_customer_features()` filters transactions to `timestamp < cutoff`. The cutoff is the simulated prediction time. This design prevents future activity from leaking into historical features.

Named aggregation

The grouped `.agg(...)` computes customer-level features and gives each result an explicit name. This is more readable than receiving hierarchical column labels and renaming later.

Join validation

Both merges use `validate="one_to_one"` because the aggregated feature tables and customer table should each contain one row per customer. If duplicates appear, Pandas raises an error instead of silently multiplying rows.

Missing-value semantics

Customers with no transactions receive zero for count, total spend, and app share. Their average, standard deviation, last transaction, and recency remain missing because zero would express a false numerical fact. The model pipeline imputes remaining numerical missing values and adds missingness indicators.

Split behavior

The data is shuffled with a fixed seed and split at customer level. Since each row already represents one customer, the same entity cannot appear in multiple partitions. For temporal prediction tasks, a time-based split would be preferable.

Model pipeline

The scikit-learn pipeline learns imputation, scaling, and category encoding from training data only. It also handles unseen validation categories. This is safer than preprocessing the complete DataFrame before splitting.

Evaluation with Pandas

Predictions are returned as a DataFrame and grouped by city. Aggregate accuracy alone may hide weak performance for a location, customer type, device, language, or other important subgroup.

9. Training / Evaluation

Pandas is not trained, but it is central to correct model training and evaluation.

9.1 Dataset preparation

1. Define the target and prediction time.
2. Define one row's observation unit.
3. Remove features unavailable at inference time.
4. Split observations before learned preprocessing.
5. Fit imputers, scalers, encoders, and feature selection only on training data.
6. Apply the fitted transformations to validation and test data.
7. Preserve identifiers separately for error analysis.

9.2 Split strategies

Data structure	Appropriate split	Leakage risk to avoid
IID rows	Random, preferably stratified	Duplicate rows across splits
Multiple rows per user	Group split by user/entity	Same entity in train and test
Time series/events	Chronological split	Future information in training features
Spatial data	Geographic/group split	Nearby correlated samples across splits
Medical data	Patient-level split	Same patient in multiple partitions

Pandas can create masks and inspect splits, while scikit-learn provides robust splitters such as `StratifiedKFold`, `GroupKFold`, and `TimeSeriesSplit`.

9.3 Metrics and reporting

Use Pandas to organize overall and subgroup metrics:

```
from sklearn.metrics import precision_score, recall_score

def group_metrics(group: pd.DataFrame) -> pd.Series:
    return pd.Series({
        "count": len(group),
        "precision": precision_score(
            group["y_true"], group["y_pred"], zero_division=0
        ),
        "recall": recall_score(
            group["y_true"], group["y_pred"], zero_division=0
        ),
    })

report = predictions.groupby("region").apply(group_metrics)
```

Metric choice belongs to the modeling problem:

- Classification: precision, recall, F1, ROC-AUC, PR-AUC, log loss.
- Regression: MAE, RMSE, R^2 , MAPE when its assumptions are acceptable.
- Ranking/retrieval: recall@k, precision@k, MRR, NDCG.
- Forecasting: MAE, RMSE, MASE, or domain-specific costs with time-based backtesting.

9.4 Overfitting and underfitting

Pandas does not overfit, but Pandas-based feature engineering can cause a model to overfit through:

- High-cardinality identifiers.
- Target encoding performed on the full dataset.
- Too many sparse one-hot features.
- Features derived from post-outcome events.
- Repeated experimentation against the test set.

Underfitting may come from destructive preprocessing, such as reducing meaningful categories to **Other**, using only global aggregates, or discarding temporal structure.

9.5 Hyperparameters

Pandas has operation parameters, not learned hyperparameters. Examples include:

- Rolling window size.
- Minimum periods in a window.
- Frequency used for resampling.
- Missing-value strategy.
- Outlier clipping thresholds.
- Category frequency cutoff.

Treat these as feature-engineering choices and select them using training/validation evidence, not test performance.

9.6 Improving pipeline quality

- Add explicit schema and range checks.
- Put transformations into reusable functions or model pipelines.
- Assert join cardinality.
- Compare row counts before and after critical operations.
- Track missingness introduced by parsing or merging.
- Make split logic reproducible.
- Run subgroup and temporal error analysis.
- Test features at prediction-time cutoffs.

10. Complexity and Cost

Let:

- n be the number of rows.
- m be the number of columns.
- g be the number of groups.
- k be the number of rows in a second table.

Approximate costs depend on dtypes, indexes, algorithms, and whether copying occurs.

Operation	Typical time	Additional memory	Notes
Column arithmetic/filter	$O(n)$	Often $O(n)$	Creates masks or result arrays
Row/column selection	$O(n)$ or less	View/copy dependent	Label lookup may use an index engine
Sort	$O(n \log n)$	Often $O(n)$	Multi-column sort has larger constants
Hash groupby	About $O(n)$ average	$O(g)$ or more	Depends on aggregation and key types
Merge/join	About $O(n + k)$ average for hash join	$O(n + k + \text{output})$	Output can explode for many-to-many keys
Drop duplicates	About $O(n)$ average	$O(n)$	Usually hash based
Rolling fixed window	Often $O(n)$	$O(n)$ result	Custom Python functions can be much slower
<code>apply(axis=1)</code>	$O(n)$ Python calls	Varies	High interpreter overhead

Operation	Typical time	Additional memory	Notes
Full correlation matrix	$O(nm^2)$	$O(m^2)$	Expensive for many numeric columns

Memory usage

A rough dense numerical table cost is:

$$\text{memory} \approx n \times m \times b$$

where b is bytes per value, plus index and object overhead. A `float64` value uses 8 bytes, so a dense $10^7 \times 20$ numerical table needs about 1.6 GB for values alone. Intermediate copies can temporarily multiply this requirement.

Use:

```
df.memory_usage(index=True, deep=True).sort_values(ascending=False)
```

to measure actual usage, including string/object contents.

CPU/GPU requirements

Standard Pandas execution is CPU-based and primarily targets one machine. Many operations are implemented in optimized native code, but arbitrary Python callbacks are slow. A GPU is normally unnecessary. For larger-than-memory or highly parallel workloads, push aggregation into a database or use a distributed/out-of-core engine.

Performance checklist

1. Read fewer rows and columns.
2. Filter and aggregate at the data source when possible.
3. Use appropriate dtypes.
4. Prefer vectorized operations.
5. Avoid repeated concatenation inside a loop; collect frames and concatenate once.
6. Avoid unnecessary `.copy()` calls, while still copying when semantic independence is required.
7. Measure before optimizing.

11. Common Use Cases

1. **Exploratory data analysis:** distributions, missingness, correlations, cohorts, and anomalies.
2. **Data cleaning:** parsing, validation, deduplication, type correction, and missing-value handling.
3. **Feature engineering:** ratios, counts, lags, windows, category features, and entity aggregates.
4. **Dataset assembly:** joining labels, profiles, events, embeddings metadata, and predictions.
5. **Time-series preparation:** resampling, aligning timestamps, computing lagged features, and backtest tables.

6. **NLP metadata processing:** document labels, text lengths, deduplication, language/source analysis, and train splits.
 7. **Computer vision metadata:** image paths, class labels, bounding-box tables, corrupted-file reports, and fold assignments.
 8. **LLM/RAG evaluation:** query, retrieved-document, answer, latency, token cost, citation, and judge-score analysis.
 9. **Experiment analysis:** comparing runs, hyperparameters, metrics, confidence intervals, and subgroup behavior.
 10. **Model monitoring:** prediction distributions, missingness drift, feature drift, latency, and error cohorts.
 11. **Reporting:** pivot tables, ranked summaries, CSV/Excel/Parquet outputs, and dashboard inputs.
 12. **Data exchange:** moving moderate-sized typed datasets among Python tools and storage systems.
-

12. Common Mistakes

12.1 Data leakage

- Computing imputation/scaling statistics on all data.
- Aggregating events that occur after prediction time.
- Using the target to create a feature without out-of-fold logic.
- Randomly splitting time-dependent or entity-dependent rows.

12.2 Chained assignment

```
# Ambiguous and unsafe
df[df["age"] < 0]["age"] = np.nan

# Explicit
df.loc[df["age"] < 0, "age"] = np.nan
```

12.3 Incorrect boolean syntax

```
# Wrong: Python scalar operator
# df[(df["age"] > 18) and (df["city"] == "Pune")]

# Correct: element-wise operator with parentheses
df[(df["age"] > 18) & (df["city"] == "Pune")]
```

12.4 Silent dtype problems

Leaving numbers as strings causes lexical sorting ("**100**" < "**20**") and breaks arithmetic. Always inspect parsing failures after coercion.

12.5 Many-to-many merge explosion

Joining on duplicated keys can multiply rows. Check key uniqueness, use `validate`, and compare row counts.

12.6 Misusing `apply`

Row-wise `apply` is often slower and less clear than vectorized arithmetic, `.str`, `.dt`, `map`, `where`, or `np.select`.

12.7 Treating missing values as ordinary equality values

```
# Wrong
df["age"] == np.nan

# Correct
df["age"].isna()
```

12.8 Applying `get_dummies` separately to splits

Training and test sets may produce different columns. Fit one encoder on training data and transform every split with it.

12.9 Losing the index unintentionally

After filtering or concatenation, indexes can be non-consecutive or duplicated. Reset only when positional row labels are desired; do not reset automatically if the index carries meaningful identity.

12.10 Dropping rows without measuring impact

`dropna()` may remove a large or biased subset. Report how many rows and which populations were removed.

12.11 Using the wrong metric or aggregation

An average of subgroup averages is not necessarily the global average. Weight appropriately. For imbalanced classification, accuracy may be misleading.

12.12 Poor validation split

Random row splitting is invalid when rows share a user, session, patient, document source, or future/past relationship.

12.13 Comparing datetimes with inconsistent time zones

Mixing timezone-aware and timezone-naive timestamps causes errors or incorrect assumptions. Normalize deliberately, commonly to UTC, then convert for presentation.

12.14 Repeated DataFrame concatenation

Appending/concatenating on every loop iteration repeatedly copies growing data. Accumulate frames in a list and call `pd.concat` once.

12.15 Unsafe deserialization

Do not load untrusted pickle files. Pickle can execute arbitrary code during deserialization.

13. Edge Cases / Limitations

Duplicate labels

Duplicate indexes or column names can make selection, alignment, reindexing, and reshaping ambiguous. Enforce uniqueness where the domain requires it.

Mixed-type columns

A column containing numbers, strings, and lists often falls back to a generic object representation. Operations become slower and semantics less predictable.

Missing-value inconsistency

`NaN`, `NaT`, `None`, and `pd.NA` have different historical behavior across dtypes and operators. Prefer nullable dtypes and test important comparisons and exports.

Floating-point precision

Binary floating-point cannot exactly represent many decimal values. Do not use ordinary floats for exact financial accounting; consider integers in the smallest currency unit or decimal-aware systems.

Very large data

Data must generally fit in memory along with intermediate results. A merge or pivot can require much more memory than the original table.

High-cardinality pivots and one-hot encoding

A pivot or dummy matrix with millions of unique categories can become enormous. Reduce cardinality, use sparse representations, or choose a different modeling strategy.

Slow Python functions

`apply`, `iterrows`, and custom rolling functions can become bottlenecks because they cross into Python for many rows.

Time-zone and daylight-saving transitions

Local times can be nonexistent or ambiguous during clock changes. Store event time in UTC and explicitly localize/convert at boundaries.

Unsorted time operations

Rolling, shifting, `merge_asof`, and resampling assume meaningful order. Sort by entity and time before computing sequential features.

Non-deterministic row order assumptions

Do not assume a merge or groupby always returns business-required ordering. Sort explicitly before display or order-sensitive operations.

Schema drift

CSV inputs can silently change column names, types, category values, or date formats. Add validation instead of trusting yesterday's schema.

Not a database or distributed engine

Pandas has no built-in transactional guarantees, concurrency model, query optimizer comparable to a database, or automatic cluster execution.

14. Variations

Here “variations” means alternative Pandas representations or neighboring execution engines.

14.1 Nullable extension dtypes

- **What changes:** Use types such as `Int64`, `boolean`, and `string` that can represent `pd.NA`.
- **When to use:** When missing values must coexist with integer/boolean/string semantics.
- **Importance:** High for placements and production data cleaning.

14.2 Categorical dtype

- **What changes:** Store repeated values as category codes plus a category vocabulary.
- **When to use:** Repeated, low-cardinality labels such as region or product type.
- **Importance:** Common interview topic for memory optimization.

14.3 MultiIndex

- **What changes:** Rows or columns have multiple index levels.
- **When to use:** Naturally hierarchical axes, grouped outputs, or panel-like data.
- **Importance:** Useful but secondary for placements; avoid it when ordinary columns are easier to maintain.

```
indexed = df.set_index(["customer_id", "timestamp"]).sort_index()
```

14.4 Sparse data

- **What changes:** Mostly-zero arrays store nonzero values compactly.
- **When to use:** Large one-hot matrices or sparse measurements.
- **Importance:** Relevant for classical ML and NLP, though scikit-learn/SciPy sparse matrices are more common for modeling.

14.5 Chunked processing

- **What changes:** Read and process bounded pieces rather than the entire file.

- **When to use:** Associative/reducible operations on data larger than comfortable memory.
- **Importance:** Strong practical interview topic. Not every task is chunkable; global sorting and arbitrary joins are harder.

14.6 SQL or DuckDB

- **What changes:** Express filtering, joins, and aggregation as queries, often without loading everything into Pandas first.
- **When to use:** Data already lives in a database, exceeds memory, or benefits from query optimization.
- **Importance:** Very high for data/ML engineering roles.

14.7 Polars

- **What changes:** Uses a columnar engine and expression-oriented eager/lazy APIs.
- **When to use:** Performance-heavy analytical transformations where team compatibility and ecosystem support are acceptable.
- **Importance:** Increasingly useful in projects; Pandas remains a core interview expectation.

14.8 Dask or Spark DataFrames

- **What changes:** Computation is partitioned, lazy, and potentially distributed.
- **When to use:** Workloads require parallel or cluster execution.
- **Importance:** High for big-data and MLOps roles. APIs resemble Pandas in places, but ordering, indexing, shuffles, and supported operations differ.

14.9 GeoPandas

- **What changes:** Adds geometry columns and geospatial operations.
- **When to use:** Maps, spatial joins, distance/region features.
- **Importance:** Specialized but valuable for geospatial ML projects.

14.10 Pandas-on-Spark-style APIs

- **What changes:** A Pandas-like interface runs on a distributed execution system.
- **When to use:** Migrating familiar analytical code to cluster-scale data.
- **Importance:** Project/industry-specific; understand that identical syntax does not guarantee identical performance.

15. Related Topics

Pandas vs NumPy

Pandas	NumPy
Labeled, heterogeneous columns	Homogeneous n-dimensional arrays
Missing/tabular/time-series tools	Core numerical array computation
Index alignment and joins	Positional broadcasting and linear algebra

Pandas frequently uses NumPy-like arrays internally and interoperates through `to_numpy()`.

Pandas vs SQL

Both filter, join, group, and aggregate. SQL excels at source-side querying, larger-than-memory data, concurrency, and database optimization. Pandas excels at interactive Python workflows, custom feature logic, and integration with ML libraries. Strong engineers combine them: reduce data in SQL, then perform model-specific shaping in Pandas.

Pandas vs Polars

Pandas offers a mature ecosystem and broad compatibility. Polars emphasizes expression-based execution, parallelism, and a columnar design. Choose based on workload, maintainability, dependency constraints, and measured performance.

Pandas vs Spark

Pandas runs on one machine and is ideal for moderate in-memory data. Spark distributes work across a cluster but introduces scheduling, shuffle, serialization, and operational overhead.

Pandas and scikit-learn

Pandas supplies labeled tables; scikit-learn supplies estimators, preprocessing transformers, splitters, and metrics. Use `Pipeline` and `ColumnTransformer` so learned preprocessing is fit only on training data.

Pandas and PyTorch

Pandas is useful for metadata and preprocessing, but PyTorch models consume tensors. Convert only model-ready numeric arrays, and use `Dataset/DataLoader` abstractions for scalable training rather than reading rows from a `DataFrame` inside every training step.

Pandas and feature stores

Pandas prototypes offline features. A feature store manages feature definitions, historical point-in-time correctness, materialization, and online/offline consistency. A Pandas feature must be reproducible at inference time to be production-safe.

Pandas and data validation

Assertions and custom checks are sufficient for small pipelines. Larger systems may use schema/data-quality tools. The core principles remain: validate types, ranges, uniqueness, nullability, freshness, and relationships.

Pandas and visualization

Pandas can prepare aggregates and exposes basic plotting integration. Matplotlib, Seaborn, Plotly, or BI tools handle richer visualization. The analytical table should be correct before it is plotted.

16. Interview Questions

1. What is Pandas, and why is it used in ML?

Pandas is a Python library for labeled tabular and time-series data. In ML it is mainly used to load, inspect, clean, join, reshape, engineer features, create splits, and analyze model outputs.

2. What is the difference between a `Series` and a `DataFrame`?

A `Series` is one-dimensional with an index and optional name. A `DataFrame` is two-dimensional with both row and column labels. `df["x"]` usually returns a `Series`; `df[["x"]]` returns a `DataFrame`.

3. What is the difference between `.loc` and `.iloc`?

`.loc` selects by labels and label-based boolean masks. `.iloc` selects by integer position. A `.loc` label slice usually includes the ending label; an `.iloc` slice excludes the ending position.

4. Why does Pandas alignment sometimes introduce `NaN`?

Pandas aligns objects by index labels. If one object contains a label missing from the other, the result for that label lacks one operand and becomes missing.

5. How do `agg`, `transform`, and `apply` differ after `groupby`?

- `agg` reduces each group to summary values.
- `transform` returns values aligned to the original row index.
- `apply` can return flexible shapes but is more general, often slower, and easier to misuse.

6. How would you handle missing values?

First measure and understand why values are missing. Options include dropping rows/columns, constant or statistical imputation, group-based imputation, interpolation for appropriate ordered data, model-native missing handling, and missingness indicators. Fit any learned imputation only on training data.

7. Why is `df[col] == np.nan` incorrect?

Missing floating-point values do not compare equal in the ordinary way. Use `df[col].isna()` or `pd.isna(...)`.

8. How do you prevent a merge from silently duplicating rows?

Check key uniqueness, state the expected cardinality through `validate=...`, compare row counts, and use `indicator=True` to inspect matched and unmatched records.

9. What is the difference between `pivot` and `pivot_table`?

`pivot` reshapes data and requires unique index-column combinations. `pivot_table` can aggregate multiple records for the same combination using functions such as mean or sum.

10. Why is vectorization faster than `iterrows()`?

Vectorized operations execute loops in optimized compiled code over contiguous arrays and avoid creating Python objects and function calls for each row. `iterrows()` has high interpreter overhead and may not preserve row dtypes as expected.

11. How would you optimize a memory-heavy DataFrame?

Read only needed columns/rows, use appropriate numeric widths, convert repeated strings to categorical or suitable string types, avoid object-heavy columns, delete unneeded intermediates, process chunks when the computation supports it, and prefer source-side aggregation. Measure using `memory_usage(deep=True)`.

12. What causes chained-assignment problems?

Two indexing operations can produce a temporary object whose relationship to the original is ambiguous. Assign once with `.loc[row_mask, column] = value`, and call `.copy()` when an independent subset is intended.

13. How would you build a seven-day rolling feature without future leakage?

Sort by entity and timestamp, group by entity, shift the value by one observation or otherwise exclude the current timestamp, then apply a time-aware or fixed rolling window. Confirm the window contains only data available before prediction time.

14. Why can applying `pd.get_dummies` separately to train and test be dangerous?

The splits may contain different categories, producing incompatible columns. Fit an encoder on training data and use it to transform every split with unknown-category handling.

15. How do `merge`, `join`, and `concat` differ?

- `merge` performs database-style joins on columns or indexes.
- `join` is a convenience method often used for index-based joining.
- `concat` stacks or aligns objects along an axis without relational key matching.

16. What is the difference between `size()` and `count()` in grouped operations?

`size()` counts rows, including rows where a selected value is missing. `count()` counts non-missing values per column.

17. Why might `mean()` and `std()` differ between Pandas and another library?

Missing-value defaults and degrees of freedom may differ. Pandas generally skips missing values in aggregations and uses `ddof=1` for sample standard deviation, while another library may use population standard deviation with `ddof=0`.

18. How do you safely parse a dirty numeric column?

Normalize known formatting, call `pd.to_numeric(..., errors="coerce")`, compare missingness before and after conversion, inspect failed raw tokens, validate ranges, and then select a suitable dtype.

```
raw = df["salary"]
cleaned = pd.to_numeric(
    raw.str.replace(",", "", regex=False).str.replace("₹", "", regex=False),
    errors="coerce",
```

```
)  
failed = raw[cleaned.isna() & raw.notna()].value_counts()
```

19. When should Pandas not be used?

Avoid it as the main engine when data and intermediates exceed memory, low-latency streaming is required, transactional/concurrent updates are needed, or a distributed engine/database can perform the workload more reliably.

20. How would you detect data leakage with Pandas?

Audit feature timestamps against prediction timestamps, inspect suspiciously high target correlations and identifier-like fields, recompute preprocessing within each split, check entity overlap across partitions, and trace every feature back to its source and availability time. Correlation alone cannot prove leakage; lineage and time semantics are decisive.

21. Explain `copy()` versus `inplace=True`.

`.copy()` requests an independent DataFrame/Series for clear ownership. `inplace=True` is not a guarantee of zero memory allocation and often makes pipelines harder to chain or reason about. Prefer explicit assignment unless mutation is genuinely clearer.

22. How would you calculate top three transactions per customer?

```
top_three = (  
    transactions.sort_values(  
        ["customer_id", "amount"], ascending=[True, False]  
    )  
    .groupby("customer_id", group_keys=False)  
    .head(3)  
)
```

23. How would you find customers present in one table but not another?

Use a left merge with `indicator=True` and filter `left_only`, or use key membership with `isin` when only existence is needed.

```
missing = (  
    orders[["customer_id"]].drop_duplicates()  
    .merge(customers[["customer_id"]], how="left", indicator=True)  
    .query("_merge == 'left_only'")  
)
```

24. What happens in a many-to-many join?

For a key appearing m times on the left and n times on the right, the join produces mn combinations for that key. This can be correct, but it must be intentional because it changes row weights and memory usage.

25. Is Pandas lazy or eager?

Traditional Pandas expressions are generally evaluated eagerly: the result is computed when the operation is called. This differs from lazy query planners that build an execution graph and optimize it before collection.

17. Practice Tasks

Task 1: Small coding task — employee analysis

Given columns `employee_id`, `department`, `salary`, `joining_date`, and `performance_score`:

1. Parse types.
2. Find invalid or missing salaries.
3. Compute department median salary.
4. Add `salary_vs_department_median` using `transform`.
5. Return the top two performers per department.

Skills tested: parsing, validation, grouping, transformation, sorting.

Task 2: Dataset project — retail customer features

Use a public retail transactions dataset.

1. Define a customer-level prediction cutoff.
2. Build recency, frequency, and monetary features.
3. Add category diversity and average basket size.
4. Create train/validation/test partitions.
5. Train a churn or repeat-purchase model.
6. Analyze errors by customer cohort.

Key challenge: prevent transactions after the cutoff from entering features.

Task 3: Experiment — vectorization benchmark

Implement the same conditional feature in three ways:

- Python loop or `iterrows()`.
- `apply(axis=1)`.
- Vectorized `np.select` or boolean assignment.

Measure time with increasing row counts and verify identical outputs. Explain why asymptotic complexity may be similar while constant factors differ greatly.

Task 4: Debugging task — exploding join

Create two tables with duplicate customer keys, perform a merge, and explain why row count increases. Add:

- Duplicate-key diagnostics.

- **validate** with the correct expected relationship.
- An aggregation or deduplication step justified by the data model.

Task 5: Analysis task — missingness investigation

For a dataset with missing values:

1. Calculate missing rate by column.
2. Compare target rate for missing versus non-missing groups.
3. Compare missingness across time and source system.
4. Decide whether to drop, impute, or preserve a missing indicator.
5. Document possible missingness mechanisms.

Task 6: Extension — reusable point-in-time features

Write a function receiving an entity table, event table, and cutoff timestamp. It should produce historical count, sum, mean, recency, and last-event features without future leakage. Add assertions for unique output keys and timestamp validity.

Task 7: Time-series task

Given hourly sensor data:

1. Parse timezone-aware timestamps.
2. Resample to hourly frequency.
3. Flag missing intervals.
4. Compute lag-1, lag-24, and prior-seven-day rolling mean.
5. Split chronologically and verify no feature uses the current target.

18. Project Ideas

Project 1: Customer Churn Feature Factory

- **What it does:** Builds point-in-time customer features from profile, transaction, and support-ticket tables; trains a churn model; generates subgroup error reports.
- **Tech stack:** Pandas, NumPy, scikit-learn, Parquet, Matplotlib/Seaborn, optional FastAPI for scoring.
- **Dataset suggestion:** IBM Telco Customer Churn plus a synthetically generated event-history table, or an online retail transaction dataset with a carefully defined repeat-purchase target.
- **Resume value:** Demonstrates relational joins, leakage prevention, temporal features, model pipelines, validation, and business-facing analysis.

Project 2: ML Data Quality and Drift Monitor

- **What it does:** Compares a reference training dataset with incoming batches and reports schema changes, missingness shifts, category changes, numerical drift, and prediction distributions.
- **Tech stack:** Pandas, SciPy/scikit-learn, Plotly or Streamlit, Parquet, scheduled Python job.
- **Dataset suggestion:** UCI Adult, credit-risk data, or a synthetic production stream with controlled drift.
- **Resume value:** Shows MLOps awareness, statistical thinking, robust aggregation, monitoring, and clear reporting.

Project 3: RAG Evaluation Analytics Workbench

- **What it does:** Analyzes one row per query-document-answer evaluation, computes recall@k, latency and cost summaries, answer-quality slices, failure cohorts, and experiment comparisons.
- **Tech stack:** Pandas, NumPy, scikit-learn metrics, Plotly/Streamlit, optional sentence-transformers for offline evaluation.
- **Dataset suggestion:** Natural Questions, HotpotQA, BEIR subsets, or a small domain-specific document collection with manually labeled relevant passages.
- **Resume value:** Connects core data engineering to modern LLM/RAG evaluation, including grouped metrics, long-to-wide reshaping, and failure analysis.

19. Quick Revision

Item	Revision note
Key idea	Labeled, vectorized, in-memory manipulation of tabular and time-series data
Main structures	<code>Series</code> and <code>DataFrame</code>
Label selection	<code>.loc</code>
Position selection	<code>.iloc</code>
Missing detection	<code>isna()</code> / <code>notna()</code>
Group summaries	<code>groupby(...).agg(...)</code>
Group feature aligned to rows	<code>groupby(...).transform(...)</code>
Relational combination	<code>merge(..., validate=..., indicator=True)</code>
Vertical/horizontal stacking	<code>pd.concat(...)</code>
Reshaping	<code>pivot</code> , <code>pivot_table</code> , <code>melt</code> , <code>stack</code> , <code>unstack</code>
Datetime tools	<code>to_datetime</code> , <code>.dt</code> , <code>resample</code> , <code>rolling</code> , <code>shift</code>
Main formula	Group mean: $\mu_k =$
When to use	Moderate-sized in-memory analysis, preprocessing, feature engineering, evaluation
Important metrics	Data metrics: missing rate, duplicate rate, join match rate, row counts; model metrics depend on task
Common traps	Leakage, index misalignment, chained assignment, dtype errors, many-to-many joins, row-wise <code>apply</code>
Interview one-liner	"Pandas is the labeled tabular layer that turns raw data into validated, leakage-safe model inputs and analyzable outputs."

Must-remember snippets

```
# Inspect
df.info()
df.isna().mean().sort_values(ascending=False)

# Filter
df.loc[(df["x"] > 0) & df["category"].isin(["A", "B"])]

# Aggregate
df.groupby("key", as_index=False).agg(total=("value", "sum"))

# Broadcast group statistic
df["group_mean"] = df.groupby("key")["value"].transform("mean")

# Safe join
df.merge(lookup, on="key", how="left", validate="many_to_one", indicator=True)

# Historical rolling feature
df["past_mean"] = (
    df.sort_values(["entity", "time"])
    .groupby("entity")["value"]
    .transform(lambda s: s.shift(1).rolling(7, min_periods=1).mean())
)
```

20. Final Cheat Sheet

Definition

Pandas is a Python library for labeled, in-memory tabular and time-series data manipulation.

Input / Output

- **Inputs:** CSV, Parquet, JSON, SQL query results, dictionaries, lists, NumPy arrays, and other tabular sources.
- **Outputs:** cleaned `Series/DataFrame` objects, NumPy-compatible matrices, analytical summaries, features, predictions, and exported files/tables.

Main steps

1. Load required data with explicit parsing.
2. Inspect shape, schema, missingness, duplicates, and ranges.
3. Clean values without hiding quality problems.
4. Join tables with key-cardinality validation.
5. Split before fitting preprocessing.
6. Engineer vectorized, point-in-time-correct features.
7. Assert invariants and export typed data.
8. Analyze model results overall and by subgroup.

Key operation parameters

Pandas has no model hyperparameters. Important operation choices include:

- `how`, `on`, and `validate` for joins.
- `dropna`, `fillna`, and imputation policy.
- `sort_values` keys and direction.
- Grouping keys and aggregation functions.
- Rolling window, `min_periods`, and whether to `shift`.
- Resampling frequency and aggregation.
- Parsing dtypes, date formats, selected columns, and chunk size.

Metrics to monitor

- Row and column counts.
- Unique-key and duplicate rates.
- Missing and invalid-value rates.
- Join match/unmatched rates.
- Category cardinality and unseen categories.
- Numerical distribution changes.
- Feature freshness and timestamp validity.
- Model metrics overall and by subgroup/time window.

Pros

- Expressive and readable tabular API.
- Excellent Python/ML ecosystem integration.
- Strong support for joins, grouping, missing data, reshaping, and time series.
- Fast vectorized operations for moderate-sized data.
- Ideal for exploration, prototypes, feature engineering, and evaluation.

Cons

- Generally limited by single-machine memory.
- Intermediate operations may make expensive copies.
- Row-wise Python functions are slow.
- Index alignment and chained assignment can surprise beginners.
- Weak fit for transactions, concurrent mutation, streaming, or cluster-scale processing.

Best use cases

- Data exploration and cleaning.
- Moderate-sized ML dataset preparation.
- Feature engineering and point-in-time analysis.
- Joining relational extracts.
- Time-series transformation.
- Experiment, prediction, and subgroup evaluation.

Final interview summary

Pandas is not a learning algorithm. It is the data-manipulation layer used to build trustworthy model inputs and interpret model outputs. Interview-ready Pandas knowledge means more than syntax: it means understanding labels, dtypes, missingness, vectorization, join cardinality, time-aware features, leakage prevention, memory cost, and validation.